

Programare Orientată Pe Obiecte

Curs 6

Polimorfism

Clase abstracte

Noiembrie 2025

Polimorfism

Un obiect al unei clase derivate are mai mult de un tip

- Un obiect al unei clase derivate are tipul clasei derivate și are și tipul clasei de bază
- Mai general, un obiect al unei clase derivate are tipul fiecăruia dintre clasele din ascendența sa
 - De aceea, un obiect dintr-o clasă derivată poate fi asignat unei variabile de tipul oricărui părinte/strămoș al său
 - Observați, totuși, că relația nu merge și invers!

Un obiect al unei clase derivate are mai mult de un tip

- Un obiect al unei clase derivate are tipul clasei derivate și are și tipul clasei de bază:
- Un obiect dintr-o clasă derivată poate fi asignat unei variabile de tipul oricărui părinte/strămoș al său, dar invers nu se poate

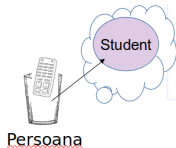
```
class Animal {  
    public void cumFace() {  
        System.out.println("Sunet");  
    }  
}  
class Caine extends Animal {  
    public void cumFace() {  
        System.out.println("Latra!");  
    }  
}  
class Pisica extends Animal {  
    public void cumFace() {  
        System.out.println("Miauna");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal();  
        Animal c = new Caine();  
        Animal p = new Pisica();  
        a.cumFace();  
        c.cumFace();  
        p.cumFace();  
    }  
}
```

Polimorfism

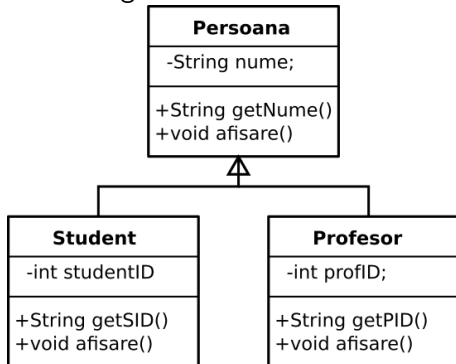
- Poli = mai multe
- Morphos = forme
- Polimorfismul se referă la această proprietate a obiectelor de a avea mai multe forme
- Spre exemplu, un obiect de tip Persoana poate referi spre un obiect de tip Student:

```
Persoana p= new Student("Ana", 1234);
```



Polimorfism

Se dă diagrama de clase:



Rezultate afișate:

Ion

1234: Ana

8: Mara

Faceți implementarea JAVA corespunzătoare diagramei de clase și discutați ce credeți că va afișa codul următor:

```
Persoana p[] = new Persoana[3];
p[0] = new Persoana("Ion");
p[1] = new Student("Ana", 1234);
p[2] = new Profesor("Mara", 8);
for(int i = 0; i < p.length; i++)
{
    p[i].afisare();
}
```

Polimorfism

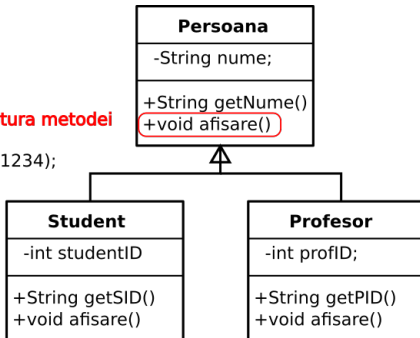
Decizii luate la compilare vs. în timpul execuției

Reguli pentru compilare:

- Compilatorul cunoaște doar tipul referință al obiectului
- Caută în clasa tipului referință dacă există metoda care se dorește a fi apelată
- Și returnează antetul metodei (semnătura)

Semnătura metodei

Persoana p = new Student("Ana", 1234);
p.afisare();



Polimorfism

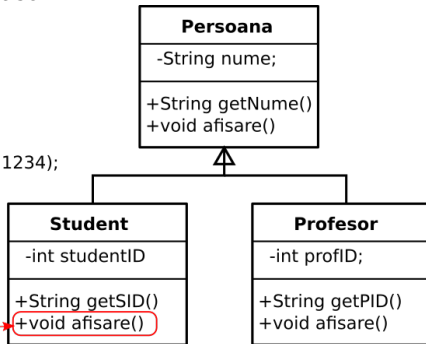
Decizii luate la compilare vs. în timpul execuției

Reguli pentru execuție:

- Se va urma tipul obiectului creat efectiv în momentul execuției
- Semnătura returnată în momentul compilării trebuie să se potrivească cu metoda din clasa actuală a obiectului
- În cazul în care metoda nu este găsită în clasa actuală, se caută mai sus în ierarhia de clase

Persoana p= new Student("Ana", 1234);
p.afisare()

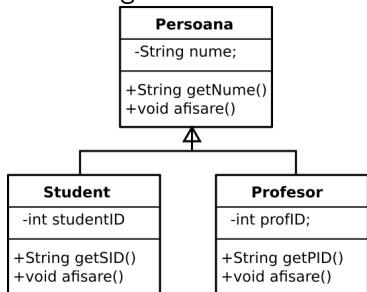
Metoda care se
executa



Polimorfism

Decizii luate la compilare vs. în timpul execuției

Se dă diagrama de clase:



Ce se întâmplă la execuția următoarelor variante de cod?

- (v1):

```
Persoana p = new Student("Ana", 1234);  
p.getSID();
```

Răspuns: Eroare de compilare

Soluție: `((Student) p).getSID();`

Pentru a evita erorile la execuție, folosiți:

```
if ( p instanceof Student ) {  
    // se executa doar daca p "este-un" Student la executia programului  
    ( (Student) s ).getSID(); }  
}
```

- (V2): `Student s = new Persoana("Ion");`

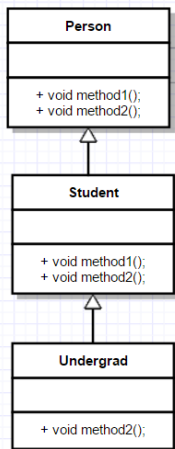
Răspuns: Eroare de compilare

Soluție: nu există

Polimorfism

Exemplu

Se dă diagrama de clase și codul:



```
public class Person {
    public void method1() {
        System.out.print("Person 1 ");
    }
    public void method2() {
        System.out.print("Person 2 ");
    }
}

class Student extends Person {
    public void method1() {
        System.out.print("Student 1 ");
        super.method1();
        method2(); //this.method2()
    }
    public void method2() {
        System.out.print("Student 2 ");
    }
}

class Undergrad extends Student {
    public void method2() {
        System.out.print("Undergrad 2 ");
    }
}
```

Polimorfism

Exemplu Cont.

```
public class Person {  
    public void method1() {  
        System.out.println("Person 1 ");  
    }  
    public void method2() {  
        System.out.println("Person 2 ");  
    }  
}  
  
class Student extends Person {  
    public void method1() {  
        System.out.println("Student 1 ");  
        super.method1();  
        method2(); //this.method2();  
    }  
    public void method2() {  
        System.out.println("Student 2 ");  
    }  
}  
  
class Undergrad extends Student {  
    public void method2() {  
        System.out.println("Undergrad 2 ");  
    }  
}
```

2

1

3

- Ce se va afișa la executarea următoarelor linii de cod?

```
Person p = new Undergrad();  
//Person p = new Student();  
p.method1();
```

Rezultate afișate:

Student 1
Person 1
Undergrad 2

Polimorfism

Exemplu Cont.

Observații:

- Se execută întâi **method1()** din clasa Student, deoarece în clasa Undergrad nu există o metodă cu această semnătură, astfel se execută prima metodă găsită mergând în sus în ierarhia de clase. Se afișează "Student 1"
- Apoi se apelează **method1()** din clasa Person (indicată de apelativul super, care în momentul compilării stabilește că apelul trebuie făcut către method1() din clasa Person). Se afișează "Person 1"
- Se apelează method2() din clasa Undergrad, deoarece compilatorul interpretează apelul "method2();" ca "**this.method2()**", unde this se referă la obiectul din care se face apelul, și anume obiectul concret creat în momentul execuției, care este de tip Undergrad

Polimorfism

Reguli pentru **this** și **super**

Reguli în ceea ce privește apelul metodelor folosind operatorii **this** și **super**:

- Când apelăm o metodă cu **super** (ex: `super.method1()`), **legarea se face la compilare**
 - Atunci se verifică care e clasa părinte
- Când apelăm o metodă cu **this** (ex. `this.method2()`), sau pur și simplu `method2()`, legarea se face în momentul execuției, în funcție de tipul concret al obiectului creat
 - Aceasta mai poartă numele de **legare dinamică**

Polimorfism

Legarea dinamică

- Apare atunci când decizia privind metoda de executat nu se poate lua decât la execuția programului
- Este nevoie de ea atunci când:
 - Variabila este declarată ca având tipul superclasei și
 - Există mai mult de o metodă polimorfică care se poate executa între tipul variabilei și subclasele sale

Cum se decide care este metoda de executat?

- 1 Dacă există o metodă concretă în clasa curentă, se execută aceea
- 2 În caz contrar, se verifică în superclasa directă dacă există acolo o metodă; dacă da, se execută
- 3 Se repetă pasul 2, verificând în sus pe ierarhie până când se găsește o metodă concretă și se execută
- 4 Dacă nu s-a găsit nici o metodă, atunci Java semnalează o eroare de compilare

Polimorfism

- O variabilă polimorfică poate părea a-și schimba tipul prin legare dinamică
- Compilatorul înțelege întotdeauna tipul unei variabile potrivit declarației
- Compilatorul permite o anumă flexibilitate prin modul de conformare la tip
- La execuție, comportamentul unui apel de metodă depinde de tipul de obiect, nu de variabilă
- Exemplu:

```
Person p;  
p = new Student();  
p = new Undergrad();  
p.method1();
```

Polimorfism

Exemplu 2:

```
public class Persoana {  
    private String name;  
    public Person(String name) {  
        this.name = name; }  
    public boolean isAsleep(int hr) {  
        return 22 < hr || 7 > hr; }  
    public String toString() {  
        return name; }  
    public void status(int hr) {  
        if (this.isAsleep(hr))  
            System.out.println("Now offline:  
                                " + this);  
        else  
            System.out.println("Now online:  
                                " + this);  
    }  
}
```

```
public class Student extends  
    Persoana {  
    public Student(String name) {  
        super(name); }  
    public boolean isAsleep(int hr)  
    { //suprascriere  
        return 2 < hr & hr < 8 ;  
    }  
  
    public static void main(String[]  
        args) {  
        Persoana p;  
        p = new Student("Vasile");  
        p.status(1);  
    }  
}
```

Rezultate afișate:

Now online: Vasile

Ordinea executiei constructorilor

Ce se va afișa la rulare?

```
class Mancare {
    Mancare() { System.out.println("Mancare()"); } }
class Paine {
    Paine() { System.out.println("Paine()"); }}
class Cascaval {
    Cascaval() { System.out.println("Cascaval()"); } }
class Salata {
    Salata() { System.out.println("Salata()"); }}
class Pranz extends Mancare {
    Pranz() { System.out.println("Pranz()"); } }
class PranzaLaPachet extends Pranz {
    PranzaLaPachet() { System.out.println("PranzaLaPachet()");
    } }
class Sandwich extends PranzaLaPachet {
    Paine b = new Paine();
    Cascaval c = new Cascaval();
    Salata s = new Salata();
    Sandwich() { System.out.println("Sandwich()"); }

    public static void main(String[] args) {
        Sandwich s = new Sandwich();
    }
}
```

De ce este util polimorfismul?

Polimorfismul permite unei superclase să rețină ceea ce este comun, lăsând specificitatea să fie tratată de subclase.

- Să presupunem că AView include o metodă calcArea:

```
public class AView {  
    public double calcArea() { return 0.0; }  
}
```

- Atunci ARectangle trebuie scris așa:

```
public class ARectangle extends AView {  
    public double calcArea() { return getWidth() * getHeight(); }  
}
```

- Iar AOval trebuie scris așa:

```
public class AOval extends AView {  
    public double calcArea() {  
        return getWidth()/2. * getHeight()/2. * Math.PI; }  
}
```

- Considerați acum:

```
public double coverageCost(AView v, double costPerSqUnit) {  
    return v.calcArea() * costPerSqUnit;  
}
```

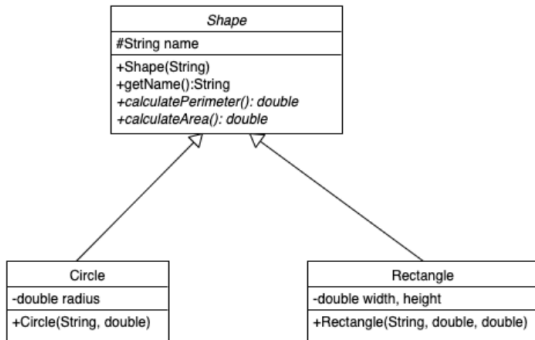
Clase abstracte

Clase abstracte

- **Definiție:** O clasă abstractă este o clasă declarată cu cuvântul cheie `abstract`, care conține cel puțin o metodă abstractă.
- Metodele abstracte sunt metode fără implementare, ce vor fi definite în subclase.
- O clasă abstractă poate conține variabile de instanță și metode concrete.
- Dintr-o clasă abstractă nu se poate instanția nici un obiect
- Fiecare subclasă a unei clase abstracte care va fi folosită pentru a instanția obiecte trebuie să ofere implementări pentru toate metodele abstracte din superclasă
- Clasele abstracte economisesc timp, deoarece nu trebuie să scriem cod "inutil" care n-ar fi executat niciodată
- O clasă abstractă poate moșteni metode abstracte
 - dintr-o interfață sau
 - dintr-o clasă
- Ele oferă o schemă generală pentru alte clase prin declararea de metode abstracte care trebuie implementate în clasele derivate.

Exemplu

Clasa abstracta Shape



- Metode abstracte in clasa Shape: `calculatePerimeter()`, `calculateArea()`
- Subclase: implementează metodele concrete `calculateArea` și `calculatePerimeter`.

Clasa abstracta Shape

Exemplu

- Să se creeze clase JAVA care modelează diferite tipuri de forme geometrice (cerc, pătrat și dreptunghi).
- Clasa Shape are o metoda abstractă `area()` care va fi implementată diferit de clasele `Circle`, `Square`, și `Rectangle` care moștenesc din Shape.
- Se va implementa o metodă `calculateTotalArea` care acceptă o listă de forme (`List< Shape >`) și calculează aria totală a tuturor formelor, fără a fi nevoie să știm tipul exact al fiecărei forme.
- Clasa Shape are o variabila statica `counter` care contorizează câte forme au fost create în cadrul aplicației.

Clase abstracte

Exemplu

```
// Clasa abstracta de baza Shape
abstract class Shape {
    String color;

    public Shape(String color) {
        this.color = color;
    }

    // Metoda abstracta pentru calcularea ariei, definita in fiecare
    // subclasa
    public abstract double area();

    // Metoda pentru afisarea detaliilor formei
    public void display() {
        System.out.println("Shape Color: " + color);
    }
}
```

Clase abstracte

Exemplu

```
class Circle extends Shape {  
    double radius;  
  
    public Circle(String color, double radius) {  
        super(color);  
        this.radius = radius;  
    }  
  
    @Override  
    public double area() {  
        return Math.PI * radius * radius;  
    }  
  
    @Override  
    public void display() {  
        super.display();  
        System.out.println("Shape Type: Circle");  
        System.out.println("Radius: " + radius);  
        System.out.println("Area: " + area());  
    }  
}
```

Clase abstracte

Exemplu

```
class Square extends Shape {  
    double side;  
  
    public Square(String color, double side) {  
        super(color);  
        this.side = side;  
    }  
  
    @Override  
    public double area() {  
        return side * side;  
    }  
  
    @Override  
    public void display() {  
        super.display();  
        System.out.println("Shape Type: Square");  
        System.out.println("Side: " + side);  
        System.out.println("Area: " + area());  
    }  
}
```

Clase abstracte

Exemplu

```
class Rectangle extends Shape {  
    double length, width;  
  
    public Rectangle(String color, double length, double width) {  
        super(color);  
        this.length = length;  
        this.width = width;  
    }  
  
    @Override  
    public double area() {  
        return length * width;  
    }  
  
    @Override  
    public void display() {  
        super.display();  
        System.out.println("Shape Type: Rectangle");  
        System.out.println("Length: " + length + ", Width: " + width);  
        System.out.println("Area: " + area());  
    }  
}
```

Clase abstracte

Exemplu

```
public class ShapeTest {  
    public static double calculateTotalArea(Shape shapes[]) {  
        double totalArea = 0;  
        for (Shape shape : shapes) {  
            totalArea += shape.area();  
        }  
        return totalArea;  
    }  
    public static void main(String[] args) {  
        Shape shapes[] = new Shape[3];  
        shapes[0]=new Circle(5, "Red");  
        shapes[1] = new Square("Blue", 4);  
        shapes[2] = new Rectangle("Green", 3, 6);  
        System.out.println("Display shapes details:");  
        for (Shape shape : shapes) shape.afisareInfo();  
        // Calculam aria totala folosind polimorfismul  
        double totalArea = calculateTotalArea(shapes);  
        System.out.print("Aria totala a formelor: " + totalArea);  
    }  
}
```

Metode Polimorfe în Constructori

- În Java, polimorfismul permite suprascrierea metodelor în subclase.
- Totuși, **apelarea metodelor polimorfe în cadrul unui constructor** poate duce la comportament neașteptat.
- Motiv: În timpul construcției, obiectul este încă în proces de inițializare, iar câmpurile din subclase pot să nu fie încă setate.

Metode Polimorfe în Constructori

Exemplu 1

```
class Parinte {
    Parinte() {
        System.out.println("Constructorul Parinte apelat");
        // Apeleaza metoda suprascrisa in clasa Copil
        afiseaza();
    }
    void afiseaza() {
        System.out.println("Afisare Parinte");
    }
}

class Copil extends Parinte {
    private String nume = "Copil";
    @Override
    void afiseaza() {
        System.out.println("Afisare Copil: " + nume);
    }
}

public class Test {
    public static void main(String[] args) {
        Copil c = new Copil();
    }
}
```

Explicație

- Constructorul clasei *Parinte* apelează metoda 'afiseaza()'.
• Rezultat: Când se face 'new Copil()', constructorul din 'Parinte' apelează metoda 'afiseaza()' a clasei 'Copil'.
• Problemă: Câmpul 'nume' nu este încă inițializat când 'afiseaza()' este apelată, ceea ce poate duce la valori 'null' sau neașteptate.

Metode polimorfice in constructori

Exemplu 2

```
abstract class Forma {
    abstract void draw();
    Forma() {
        System.out.println("Forma() inainte de draw()");
        draw();
        System.out.println("Forma() dupa draw()");
    }
}

class FormaRotunda extends Forma {
    private int raza = 1;
    FormaRotunda(int r) {
        raza = r;
        System.out.println("FormaRotunda.FormaRotunda(), raza = " + raza);
    }
    void draw() {
        System.out.println("FormaRotunda.draw(), raza = " + raza);
    }
}

public class PolyConstructors {
    public static void main(String[] args) {
        new FormaRotunda(5);
    }
}
```

Metode polimorfice in constructori

Exemplu 2

Se va afisa:

- Forma() inainte de draw(),
- FormaRotunda.draw(), raza = 0,
- Forma() dupe draw()",
- FormaRotunda.FormaRotunda(), raza = 5

Ordinea în care se execută inițializările

Ordinea execuției când avem moștenire este următoarea:

- 1 Inițializarea câmpurilor statice și a blocurilor statice ale clasei părinte (o singură dată, când clasa părinte este încărcată).
- 2 Inițializarea câmpurilor statice și a blocurilor statice ale clasei derivate (o singură dată, când clasa derivată este încărcată).
- 3 Inițializarea câmpurilor de instanță și a blocurilor de inițializare a instanței din clasa părinte (pentru fiecare instanță creată).
- 4 Executarea constructorului clasei părinte.
- 5 Inițializarea câmpurilor de instanță și a blocurilor de inițializare a instanței din clasa derivată.
- 6 Executarea constructorului clasei derivate.

Ordinea in care se execută inițializările

Exemplu 3

```
public abstract class Forma {
    String culoare = "red";
    abstract void draw();
    Forma() {
        System.out.println("Forma inainte de draw");
        draw();
        System.out.println("Forma dupa draw");
    }
}

public class FormaRotunda extends Forma{
    private int raza = 1;
    FormaRotunda(int r) {
        raza = r;
        System.out.println("Constructor FormaRotunda raza = " + raza);
    }
    void draw() {
        System.out.println("Draw Forma rotunda, raza = " + raza + "
            Culoarea " + culoare);
    }
}

public class Main {
    public static void main(String[] args) { new FormaRotunda(5); }
}
```

Concluzii si Exercitii Propuse

Concluzii

Restricții:

- O clasă abstractă nu poate fi instanțiată direct.
- Apelarea unei metode abstracte fără a fi implementată într-o subclasă va genera o eroare de compilare.
- Exemplu: `Shape s = new Shape();` // Eroare de compilare

Clase Abstracte vs. Interfețe

- Clasele abstracte pot conține metode concrete și variabile de instanță.
- Interfețele definesc doar metode fără implementare (până la Java 8).
- În Java 8+, interfețele pot conține metode implicite (`default`).
- O clasă poate extinde doar o singură clasă abstractă, dar poate implementa mai multe interfețe.

Concluzii

Necesitatea Utilizării Claselor Abstracte

Avantaje

- Structură comună și reutilizabilă.
- Suport pentru polimorfism.

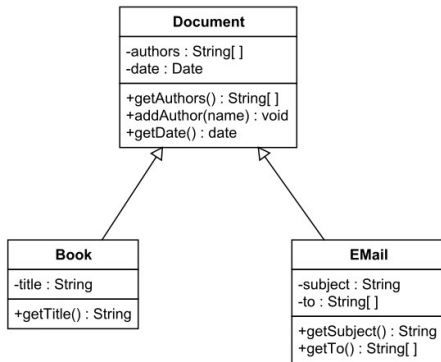
Dezavantaje

- Nu permite moștenirea multiplă.
- Poate complica ierarhiile de clase.
- Clasele abstracte sunt utile pentru crearea unui model de bază.
- Oferă o structură comună pentru clase derivate fără a impune implementarea completă.
 - Oferă flexibilitate în definirea comportamentului diverselor clase.
- Permit polimorfismul, unde obiectele de diferite clase pot fi tratate uniform.

Exercitii Propuse(1)

Realizati ierarhia de clase din figura alăturată.

- 1 Într-o clasă cu metoda main() instantiati un document de tip Book si un document de tip Email.
- 2 Adaugati la clasa Document o metoda de afisare a informatiilor despre Document (autori, data).
- 3 Adaugati in clasa Document o metoda abstracta (in plus fata de cele 3 metode). Discutati cu colegii ce efect are aceasta adaugare a metodei abstracte asupra clasei !



Exerciții Propuse(2)

Realizați clasele Java care pentru un sistem de gestionare a formularelor (ex. contact, înscriere, feedback). Includeți următoarele clase:

- **Clasa abstractă Formular:**

- Atribute:

- String titlu - titlul formularului.
- Map<String, String> campuri - câmpurile formularului, unde cheia este numele câmpului, iar valoarea este conținutul acestuia.

- Metode:

- Metodă abstractă validare() - validarea datelor specifice.
- Metodă afisareFormular() - afișează titlul și toate câmpurile formularului.

- **Clasa FormularContact:**

- Extinde Formular.
- Atribute specifice: nume, email, mesaj.
- Implementarea metodei validare():
 - Validează formatul email.
 - Verifică că nume și mesaj nu sunt goale.

Exerciții Propuse(2)

● Clasa **FormularInscriere**

- Extinde `Formular`.
- Atribute specifice: `nume`, `prenume`, `dataNasterii`, `telefon`.
- Implementarea metodei `validare()`:
 - Verifică că `nume` și `prenume` nu sunt goale.
 - Validează formatul pentru `dataNasterii` și `telefon`.

● Clasa **FormularFeedback**

- Extinde `Formular`.
- Atribute specifice: `nume`, `rating`, `comentariu`.
- Implementarea metodei `validare()`:
 - Verifică că `rating` este între 1 și 5.
 - Verifică că `comentariu` nu depășește 500 de caractere.

● Clasa **GestiuneFormulare**: Gestionează lista de formulare și permite adăugarea și validarea acestora.

- Atribute: `List<Formular> formulare`
- Metode:
 - `adaugareFormular(Formular f)` - adaugă un formular.
 - `validareFormulare()` - iterează și validează fiecare formular.
 - `afisareToateFormularele()` - afișează toate formularele.

Exemplu de Utilizare

```
public class TestFormulare {  
    public static void main(String[] args) {  
        GestiuneFormulare gest = new GestiuneFormulare();  
        Formular contact = new FormularContact("Contact", "Ion Popescu",  
            "ion.popescu@example.com", "Salut!");  
        Formular inscriere = new FormularInscriere("Inscriere", "Ana",  
            "Ionescu", "2000-05-10", "0723456789");  
        Formular feedback = new FormularFeedback("Feedback", "Mihai",  
            4, "multumit!");  
        gest.adaugareFormular(contact);  
        gest.adaugareFormular(inscriere);  
        gest.adaugareFormular(feedback);  
  
        gest.validareFormulare();  
        gest.afisareToateFormularele();  
    }  
}
```